

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:30

Annexure A

CLASS TEST

Code of Conduct:

I K.Madhulika(192524190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Madhulika**

Q. No	Understanding of Concepts (25)	Explanation (25)	Application (20)	Organization (15)	Accuracy (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Annexure A

CLASS TEST

Q1. Consider the following while-statement:

while $X < Y$ or $Y \leq 3 * X + 2$ do

$X := X + 1$;

- (i). Construct the parse tree for the above given while-statement
- (ii). Write the intermediate code for the while-statement
- (iii). Explain how the Boolean condition is evaluated during execution
- (iv). Analyze the role of conditional and unconditional jumps in the generated code

Q2. Write down the translation scheme to generate code for assignment statements involving arithmetic operators. List the scheme for generating three address code for the assignment statement:

$P = (m - n) * (x + y) / z$

- (i). Construct the syntax-directed translation scheme
- (ii). Generate the corresponding three address code
- (iii). Show the temporary variables used during evaluation
- (iv). Analyze how operator precedence affects code generation

Q3. Consider the arithmetic expression:

$a * b + c / d \uparrow e - f * g$

- (i). Analyze the given expression and explain how precedence and associativity determine the order of evaluation
- (ii). Construct the Three Address Code (TAC) using temporary variables
- (iii). Represent the generated TAC in the following forms:
 - (a). Quadruples
 - (b). Triples
 - (c). Indirect Triples
- (iv). Compare the above representations in terms of optimization efficiency, memory utilization, and ease of rearranging instructions during compiler optimization

Q4. Construct three address code using the following production for Boolean expression:

$(a < b)$ or $(c > d)$ and $(e \leq f)$

$B \rightarrow B1 \parallel B2$

$B \rightarrow B1 \ \&\& \ B2$

$B \rightarrow ! \ B1$

$B \rightarrow E1 \ \text{relop} \ E2$

$B \rightarrow \text{true}$

$B \rightarrow \text{false}$

- (i). Generate the three address code using short-circuit evaluation
- (ii). Show the labels and jump statements clearly
- (iii). Explain how backpatching is applied during Boolean expression translation
- (iv). Analyze the role of true-list and false-list in intermediate code generation

Q5. Consider the following code segment written in a high-level programming language:

```
if (m < n) {  
    p = q * r;  
}  
else {  
    p = q + r;  
}
```

Using the backpatching technique for intermediate code generation, perform the following tasks:

- a). Generate the intermediate code for the above code segment using three-address code format
- b). Show the process of backpatching the jump statements to handle the if-else conditions correctly
- c). Assume that the variables m, n, p, q, and r are stored in memory locations and intermediate labels are represented as L1, L2, L3, etc.
- d). Explain how control transfers between THEN part, ELSE part, and exit statement during execution.

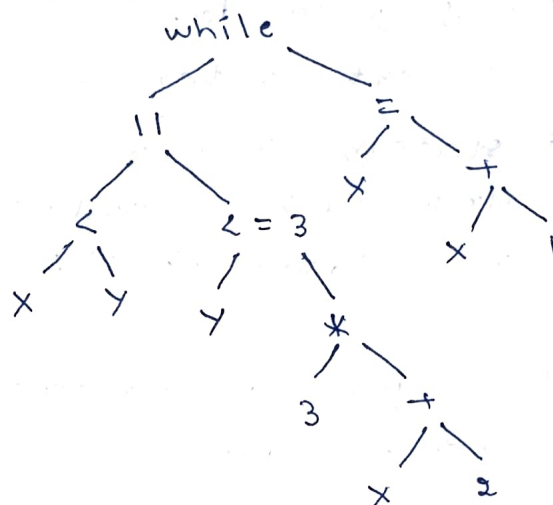
ORUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

CSA 1419 - Compiler Design

Name: K. Madhulika

Regno: 192524190

1. Parse tree:while $x < y$ or $y \leq 3 * x + 2$ do $x = x + 1$ Three Address code:

1: if $x < y$ goto 3
 goto 2

2: if $t_1 = 3 * x$ goto 3
 $t_2 = t_1 + 2$
 $y \leq t_2$

goto 4

3: $t_3 = x + 1$
 $x = t_3$
 goto 4

4:

Given, condition is

while $(X < Y)$ or $Y \leq 3 * X + 2$

do $X = X + 1$

1. or is having the highest priority. So we evaluate from left to right.

2. First $X < Y$ is evaluated. If it is true it will directly go to do. If it is false it will go to next statement

3. In second statement $*$ have highest priority so first $3 * X$ is evaluated and then added to 2 later checks less than or equal to Y .

4. If it is true goto do statement. If it is also false exit the loop.

2. Given, $p = (m - n) * (x + y) / z$

Syntax directed translation scheme:

$E \rightarrow E_1 + E_2$	$E.val = E_1.val + E_2.val$
$E \rightarrow E_1 - E_2$	$E.val = E_1.val - E_2.val$
$E \rightarrow E_1 * E_2$	$E.val = E_1.val * E_2.val$
$E \rightarrow E_1 / E_2$	$E.val = E_1.val / E_2.val$
$E \rightarrow (E)$	$E.val = (E.val)$
$E \rightarrow id$	$E = lex digit.id$

Three Address code:

$$t_1 = m - n$$

$$t_2 = x + y$$

$$t_3 = t_1 * t_2$$

-	*
---	---

 $abxcde\uparrow/+fg$
 $- < *$
 $abxcde\uparrow/+fg*-$

postfix expression = $abcxcde\uparrow/+fg*-$

Three Address code:

$$t_1 = a * b$$

$$t_2 = c / d$$

$$t_3 = f * g$$

$$t_4 = t_1 + t_2$$

$$t_5 = e - t_3$$

$$t_6 = t_4 \uparrow t_5$$

Quadruples:

operator	arg (1)	arg (2)	result
*	a	b	t ₁
↑	t ₄	t ₅	t ₆
/	c	d	t ₂
+	t ₁	t ₂	t ₄
*	f	g	t ₃
-	e	t ₃	t ₅

Triples:

	op.	arg (1)	arg (2)	
0	*	a	b	
1	↑	3	5	
2	/	c	d	
3	+	0	2	

$$t_4 = t_3 / z$$

$$p = t_4$$

t_1, t_2, t_3, t_4 are the temporary variables used during evaluation.

t_1 is used to store $m-n$

t_2 is used to store $x+y$

t_3 is used to store $(m-n) \times (x+y)$

t_4 is used to store $(m-n) \times (x+y) / z$

Finally it is stored in p .

→ Brackets have highest priority. First statements inside brackets are evaluated.

→ $*$ and $/$ have equal priority so, they are evaluated from left to right.

3. Given, $a \times b + c / d \uparrow e - f \times g$

infix to postfix:

*	
---	--

ab

$* > +$

+	
---	--

$ab \times c$

$+ < /$

+	/	
---	---	--

$ab \times cd$

$/ < \uparrow$

+	/	$\uparrow$	
---	---	------------	--

$ab \times cde$

$\uparrow > -$

+	/	
---	---	--

$ab \times cde \uparrow$

$/ > -$

+	
---	--

$ab \times cde \uparrow /$

$+ = -$

-	
---	--

$ab \times cde \uparrow / + f$

4	*	f	g	
5	-	e	4	

Indirect :

	statements		op	arg (1)	arg (2)
0	14	14	*	a	b
1	15	15	↑	17	19
2	16	16	/	c	d
3	17	17	+	14 14	16 16
4	18	18	*	f	g
5	19	19	-	e	18

4. Given, $(a < b)$ or $(c > d)$ and $(e \leq f)$,

$B \rightarrow B_1 \parallel B_2$

$B \rightarrow B_1 \&\& B_2$

$B \rightarrow ! B_1$

$B \rightarrow B_1 \text{ xlop } B_2$

$B \rightarrow \text{True}$

$B \rightarrow \text{False}$

Three Address code :

1: if $(a < b)$ goto 4
goto 2

2: if $(c > d)$ goto 3
goto 5

3: if ($e \leq f$) goto 4

goto 5

4: True

goto 6

5: False

6:

1. Here, between and and or, and have highest priority. so, first and operation is performed.

2. Then, or operation is performed.

5. Given, if ($m < n$) {

$P = q * r$;

}

else {

$P = q + r$;

}

Three Address code :

1: if $m < n$ goto 2
goto 3

2: $t_1 = q * r$

$P = t_1$

goto 4

3: ~~if~~ $t_2 = q + r$

$P = t_2$

4 :

If L_1, L_2, L_3, L_4 are used instead for m, n, p, q

if ($L_1 < L_2$) {

$L_3 = L_4 * L_5;$

}

else {

$L_3 = L_4 + L_5;$

}

Three Address code:

1 : if $L_1 < L_2$ goto 2

goto 3

2 : $t_1 = L_4 * L_5$

$L_3 = t_1$

goto 4

3 : $t_2 = L_4 + L_5$

$L_3 = t_2$

4 :

1. First if statement is evaluated.

2. If if statement is false then go to else statement.

3. Execute else statement and exit from the loop.